

Multiplicador Booth Wallace

Contreras R Orlando^{1,2*} and Garcia B Iker^{1,2*†}

³Departamento de Sistemas Electrónicos, UAA, Av. Universidad 940,
Aguascalientes, 20131, Aguascalientes, México.

*Corresponding author(s). E-mail(s): [al348390,al307630}@edu.uaa.mx](mailto:{al348390,al307630}@edu.uaa.mx);

†Ambos autores contribuyeron de forma equitativa a este trabajo.

Abstract

El presente documento busca implementar la multiplicación de la forma más efectiva, para ello se usara el codificamiento de Booth en Radix 2, e implementar la suma mediante Carry save's siguiendo la estructura del Arbol de Wallace

Keywords: VHDL, FPGA, Booth, Multiplicador, Carry Save, Multiplicador con Signo

1 Introducción

Este multiplicador de 8x8 combina la eficiencia del algoritmo de Booth para la codificación de los operandos con el uso de sumadores en paralelo tipo Wallace Tree para la reducción rápida de productos parciales. Esta arquitectura permite optimizar la multiplicación de números con signo al reducir la cantidad de operaciones necesarias y mejorar el rendimiento general del sistema. Su diseño representa una aproximación avanzada en la implementación de multiplicadores aritméticos dentro del ámbito del diseño digital.

La multiplicación eficiente es un componente esencial en una amplia gama de aplicaciones como procesamiento digital de señales, visión por computadora, aprendizaje automático y sistemas embebidos. En estos casos, es necesario ejecutar operaciones aritméticas complejas con alta velocidad y precisión, especialmente cuando se trata de datos en formato binario con signo. En este sentido, el multiplicador 8x8 basado en la técnica Booth-Wallace se presenta como una solución eficiente y escalable para tareas de procesamiento intensivo en hardware digital.

2 Metodología

2.1 Booth

Para poder implementar este codificado fue necesario manejar las entradas de diferente forma; en multiplicadores pasados hacíamos directamente el producto multiplicando cada término, en este no, lo que se hará es tomar un vector como el de control y otro como el que se irá sumando/restando según sea el caso, como se puede apreciar en 1. Cabe hacer mención que para restar el valor deberemos aplicar complemento a dos extendiendo a 16 bits siendo nuestra entrada de 8 bits.

El método de Booth establece que debe analizarse en bits al vector de control, en este caso para el RADIX-2 se analizará por pares, además de esto, es necesario añadir 2 bits extra con el valor de 0, uno al inicio y otro al final, esto para considerar esos dos casos. Se contará con varios acumuladores, que serán nuestros productos parciales.

Después de cada operación se deberá ir recortando el bit y rellenarse con 0's, para poder apreciar esto de una mejor forma se puede ver en el siguiente esquemático, siendo la estructura *generate* con valores de 0 a 8, se puede ver como cada valor se irá almacenando en una memoria/*array*. Donde cada uno será un vector de tamaño de 16 bits, es decir, se tendrá un vector de vectores, siendo el tamaño de la memoria de 6 vectores. En la figura 1 se puede apreciar la arquitectura construida.

Table 1 Casos del algoritmo de Booth radix-2

Q_1	Q_0	Acción	Operación
0	0	Nada	$A = x0000$
0	1	Sumar M	$A = M$
1	0	Restar M	$A = -M$
1	1	Nada	$A = x0000$

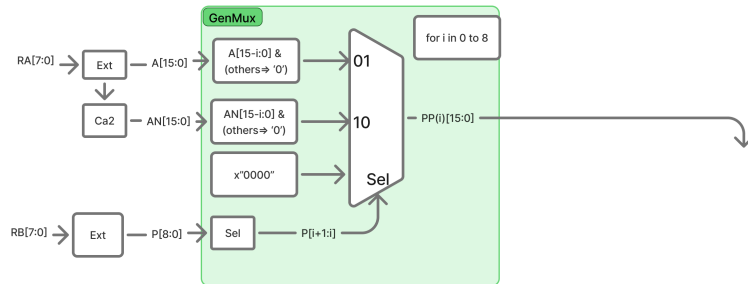


Fig. 1 Diseño de Booth

2.2 Arbol de Wallace

Una vez teniendo nuestros productos parciales, procederemos a sumarlos. Como se explicó previamente, se creó un vector llamado PP donde se almacenan vectores, siendo cada vector un Producto Parcial. Para poder sumar de forma efectiva implementaremos el carry save que prácticamente son half adder concatenados sin propagar el carry entre ellos, cada bloque de Carry Save tiene 3 entradas, y tendrá dos salidas, la salida para Carry y la salida para la suma, en el último bloque se usará un propagador de carry, puede usarse un full adder sin problema, se utilizó un carry look ahead (CLA) como optimización de tiempos. En la arquitectura se instanciaron 7 bloques, se puede apreciar toda la arquitectura en la figura 2.

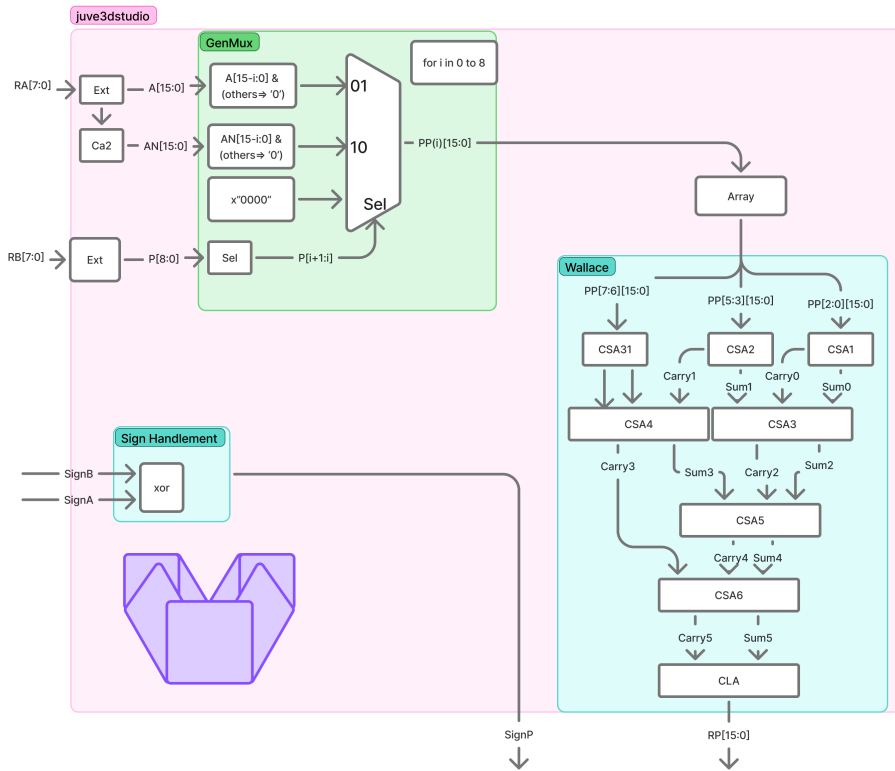


Fig. 2 Arquitectura completa

4 Conclusiones

Lo más desafiante de esta práctica fue investigar y comprender el funcionamiento del algoritmo de Booth y los carry-save adders de Wallace, así como su correcta implementación en código, asegurándonos de que no existieran casos que pudieran fallar. Uno de los principales retos surgió al manejar el valor de entrada FF, ya que en ese caso solo se generaba un producto parcial distinto de cero, lo que indicaba un comportamiento incorrecto. La solución consistió en agregar un bit adicional al final, lo que permitió que el algoritmo funcionara correctamente y resolviera el problema.

5 Anexos

Esto es la parte más importante de los RTL, si se gusta observar con mayor detalle, los PDFs estarán en el documento adjunto. Asimismo se podrá observar el código actualizado en: [GitHub Repository](#)

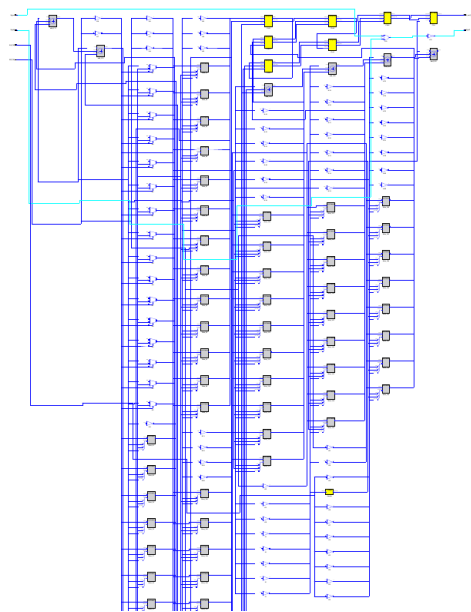


Fig. 6 RTL del Componente principal

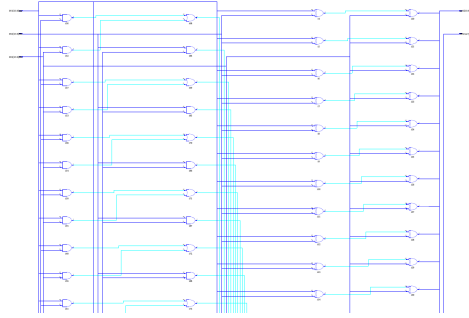


Fig. 7 RTL del CSA

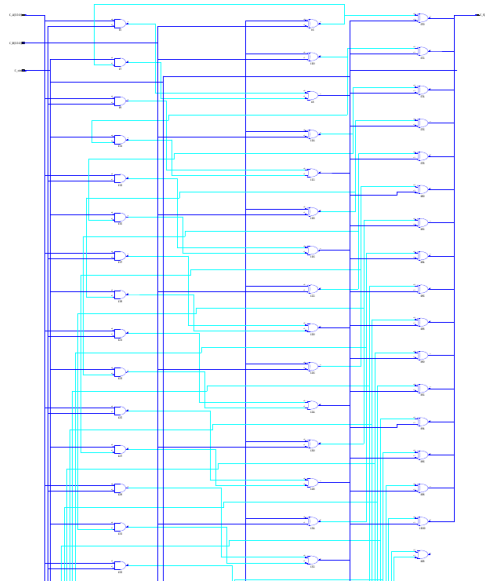


Fig. 8 RTL del Componente CLA

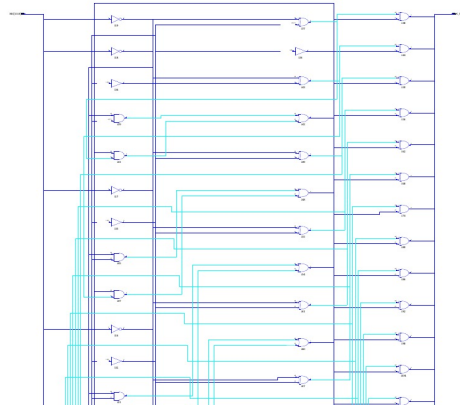


Fig. 9 RTL del Componente Complemento a 2

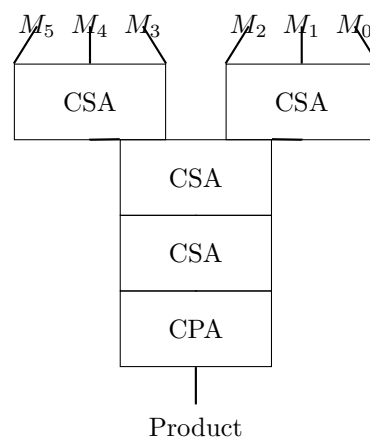


Fig. 10 Wallace-tree multiplier core (6-bit example)